期中考解答（B卷，2026/11）

本解答以課程 lab（Ch01–Ch06 中文 lab）的寫法為主；執行環境對齊電腦教室 `packages.txt`（Python 3.9.13、sklearn 1.6.1、statsmodels 0.13.2、ISLP 0.4.0）。

題目一：使用 "Data1" 資料集進行迴歸分析

請使用 **"Data1"** 資料集，其中應變數為 y，自變數為數值變數 $V_1, \ldots, V_8$ 與**類別變數** V_9（取值為 `G1`、`G2`、`G3`）。請回答下列問題：

- 當使用具有隨機性的演算法（例如隨機抽樣）時，請設定 `random_state=2026` 以確保結果可重現。
- 若所選模型**不具尺度不變性（scale-invariance）**，請在建模前對輸入的自變數進行標準化（standardization）。

(a) 以盒鬚圖（boxplot）繪製 y 在 V_9 各組的分布，並分別繪製 y 對 V_1、V_2、V_3 的迴歸圖（regression plot，即散佈圖加上簡單線性迴歸的擬合直線）。V_9 的各組之間 y 是否有差異？y 與 V_1、V_2、V_3 的關係是否都是線性的？ (6%)

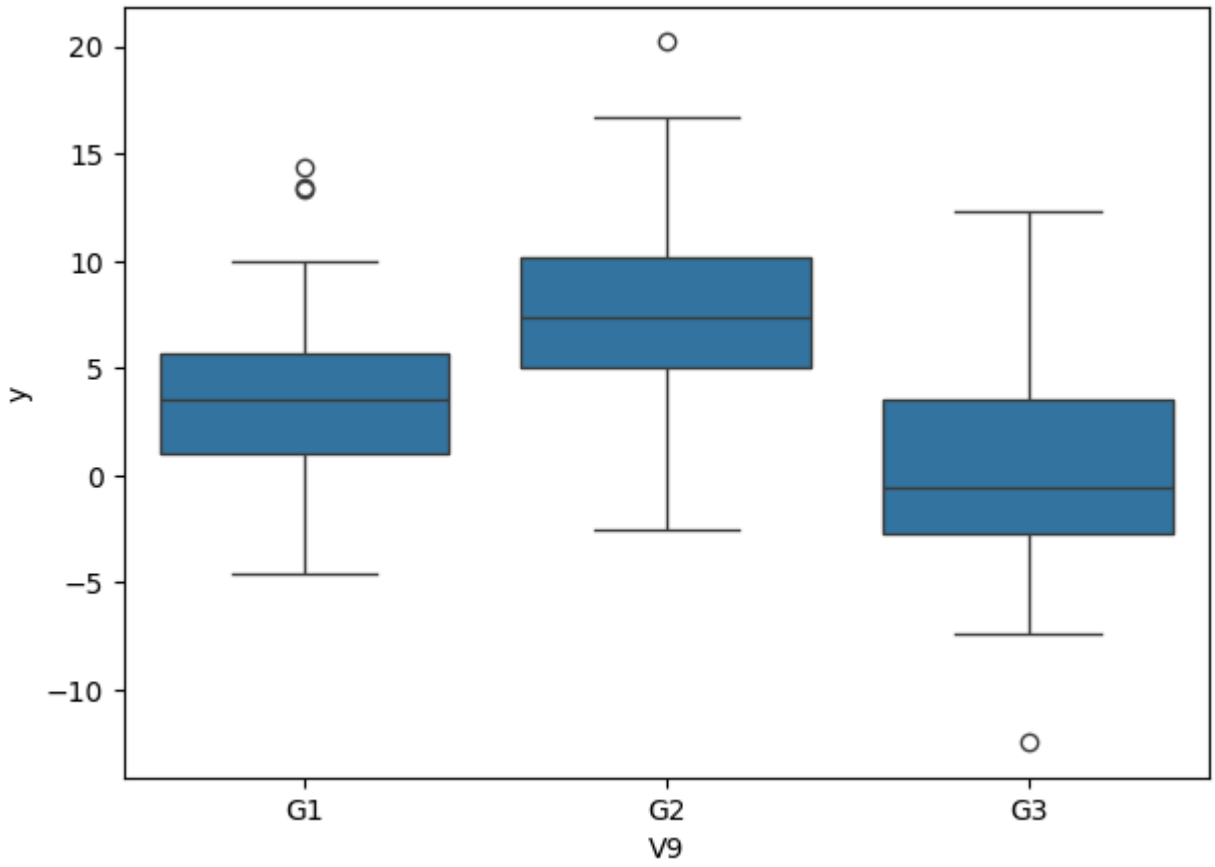
```
df1 = pd.read_csv('Data1.csv')
df1.describe(include='all')
```

	V1	V2	V3	V4	V5	V6	V7	
count	120.000000	120.000000	120.000000	120.000000	120.000000	120.000000	120.000000	120
unique	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
top	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
freq	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
mean	-0.004343	-0.064401	0.001812	-0.158503	-0.036713	0.089741	-0.012722	0
std	1.033448	1.037160	1.121052	1.124341	1.094137	1.129565	0.963818	0
min	-2.251000	-2.190800	-3.231000	-2.732100	-3.622900	-3.571200	-2.271200	-1
25%	-0.661450	-0.770075	-0.652775	-1.072650	-0.807575	-0.634325	-0.616825	-0
50%	0.001700	-0.190750	0.051100	-0.211300	0.030900	0.074500	0.009900	0
75%	0.713475	0.593975	0.829625	0.790525	0.743150	0.857525	0.569800	0
max	3.312700	2.411700	2.562100	2.997200	2.476400	3.368500	3.030500	2

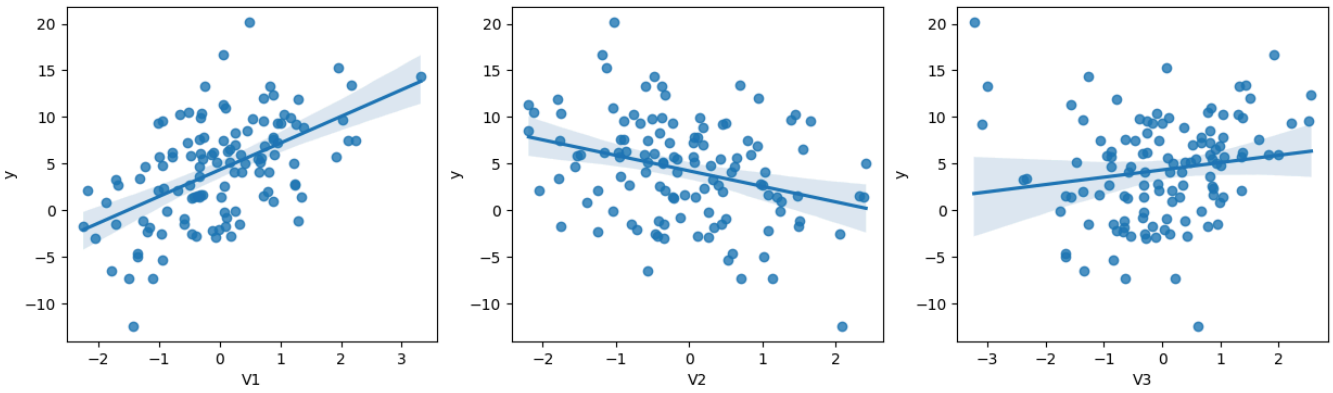
```
fig, ax = subplots(figsize=(7, 5))
sns.boxplot(x='V9', y='y', data=df1, order=['G1', 'G2', 'G3'], ax=ax);
R('a_means', df1.groupby('V9')['y'].mean().round(2).tolist(), 2)
df1.groupby('V9')['y'].mean()
```

V9 各組 y 平均 (G1, G2, G3)：3.66, 7.74, 0.25

```
V9
G1    3.660648
G2    7.738635
G3    0.247491
Name: y, dtype: float64
```




```
fig, axes = subplots(1, 3, figsize=(15, 4))
for ax, v in zip(axes, ['V1', 'V2', 'V3']):
    sns.regplot(x=v, y='y', data=df1, ax=ax)
```




```
fit_v3 = sm.OLS(df1['y'], sm.add_constant(df1['V3'])).fit()
R('a_v3slope', fit_v3.params['V3'], nd=2)
R('a_v3p', fit_v3.pvalues['V3'], nd=3)
```

```
y 對 V3 簡單迴歸斜率：0.79
V3 斜率 p 值：0.083
```

```
0.08254600110605174
```

> **參考答案**：三組的 y 平均分別約為 {G1, G2, G3} = 3.66, 7.74, 0.25，`G2` 明顯較高、`G3` 較低，V_9 與 y 有關。y 對 V_1（正）、V_2（負）的迴歸圖中，點大致落在擬合直線兩側，

> 呈線性關係；y 對 V_3 的迴歸直線斜率小且不顯著（約 0.79，p 值約 0.083），看不出明確的線性趨勢，且兩端的點略為偏高。因為 y 同時受其他變數影響，邊際的迴歸圖不容易看出 V_3 的非線性，(c) 的殘差圖會更清楚。

(b) 將 V_9 轉為虛擬變數（dummy variables，以 `G1` 為基準組），得到 `V9_G2`、`V9_G3`（若 statsmodels 出現 object dtype 錯誤，請將虛擬變數轉為數值型 0/1）。使用 `train_test_split` 以 `test_size=0.25` 切分資料，並以訓練集擬合包含全部自變數與截距的多元線性迴歸模型。請解釋 `V9_G2` 與 `V9_G3` 的係數意義，它們是否顯著？並回報此模型的 adjusted R^2。 (8%)

```
X = pd.get_dummies(df1.drop(columns='y'), columns=['V9'], drop_first=True, dtype=float)
y = df1['y']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=2026)
X.columns.tolist()
```

```
['V1', 'V2', 'V3', 'V4', 'V5', 'V6', 'V7', 'V8', 'V9_G2', 'V9_G3']
```

```
design = MS(X.columns).fit(X_train)
results = sm.OLS(y_train, design.transform(X_train)).fit()
summarize(results)
```

	coef	std err	t	P>|t|
intercept	3.3069	0.521	6.349	0.000
V1	3.1724	0.336	9.455	0.000
V2	-2.1826	0.301	-7.255	0.000
V3	1.0176	0.265	3.835	0.000
V4	-0.7638	0.302	-2.533	0.013
V5	0.0821	0.287	0.286	0.776
V6	-0.3657	0.271	-1.351	0.180
V7	-0.4044	0.329	-1.230	0.222
V8	-0.3179	0.334	-0.953	0.343
V9_G2	4.2219	0.715	5.901	0.000
V9_G3	-2.0405	0.812	-2.514	0.014

```
R('b_g2', results.params['V9_G2'], 2); R('b_g3', results.params['V9_G3'], 2)
R('b_pg2', '%.1e' % results.pvalues['V9_G2']); R('b_pg3', '%.1e' % results.pvalues['V9_G3'])
R('b_adj', results.rsquared_adj)
```

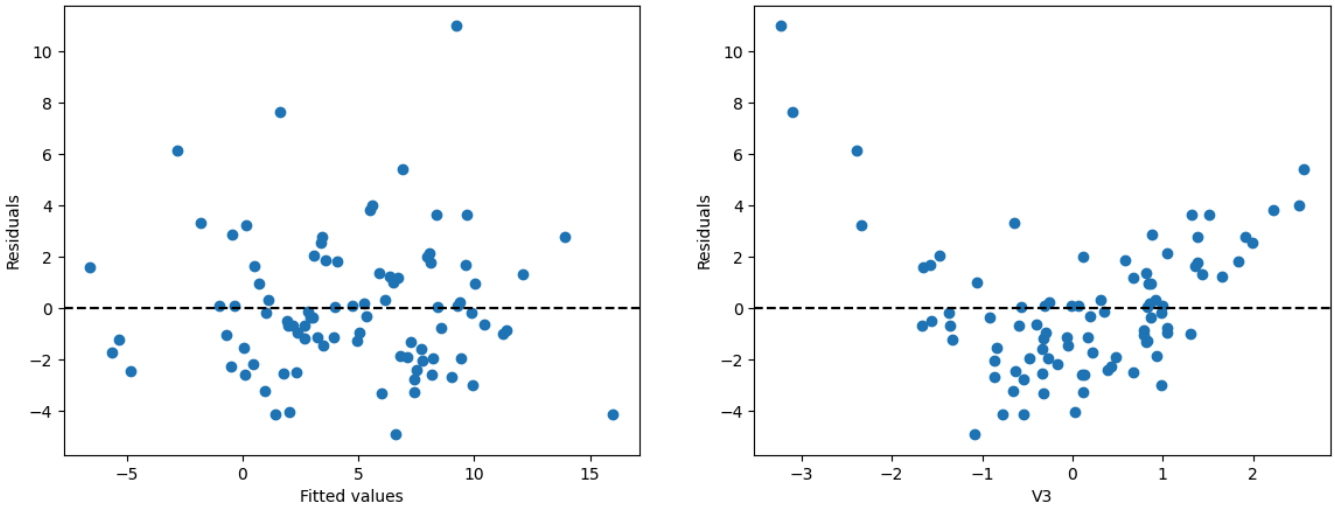
```
V9_G2 係數：4.22
V9_G3 係數：-2.04
V9_G2 p 值：8.6e-08
V9_G3 p 值：1.4e-02
(b) 模型 adjusted R²：0.702
```

```
0.7019124626202762
```

> **參考答案**：`V9_G2` 的係數約 4.22（p = 8.6e-08）：在其他變數固定時，`G2` 組的 y 平均比基準組 `G1` 高約 4.22；`V9_G3` 的係數約 -2.04（p = 1.4e-02），`G3` 組比 `G1` 低。兩者都顯著。此模型的 adjusted R^2 = 0.702。

(c) 繪製 (b) 模型的殘差對擬合值（residuals vs fitted values）圖，以及殘差對 V_3 的散佈圖，描述殘差的形狀。接著加入 V_3^2（新增欄位 `V3sq = V3**2`）重新擬合。V_3^2 的 p 值為何？adjusted R^2 與測試集 MSE 如何變化？**以下 (d)–(f) 的候選自變數為 V_1–V_8、`V9_G2`、`V9_G3`、`V3sq`，共 11 個。** (10%)

```
fig, axes = subplots(1, 2, figsize=(14, 5))
axes[0].scatter(results.fittedvalues, results.resid)
axes[0].axhline(0, c='k', ls='--'); axes[0].set_xlabel('Fitted values'); axes[0].set_ylabel
('Residuals')
axes[1].scatter(X_train['V3'], results.resid)
axes[1].axhline(0, c='k', ls='--'); axes[1].set_xlabel('V3'); axes[1].set_ylabel('Residual
s');
```




```
X_train2, X_test2 = X_train.copy(), X_test.copy()
X_train2['V3sq'] = X_train2['V3']**2
X_test2['V3sq'] = X_test2['V3']**2
design2 = MS(X_train2.columns).fit(X_train2)
results2 = sm.OLS(y_train, design2.transform(X_train2)).fit()
mse1 = mean_squared_error(y_test, results.predict(design.transform(X_test)))
mse2 = mean_squared_error(y_test, results2.predict(design2.transform(X_test2)))
R('c_p', '%.1e' % results2.pvalues['V3sq']); R('c_coef', results2.params['V3sq'], 2)
R('c_adj', results2.rsquared_adj); R('c_mse1', mse1, 2); R('c_mse2', mse2, 2)
summarize(results2)
```

```
V3sq 的 p 值：1.0e-22
V3sq 係數：1.22
加入 V3sq 後 adjusted R²：0.913
(b) 線性模型測試 MSE：10.55
加入 V3sq 後測試 MSE：2.55
```

	coef	std err	t	P>|t|
intercept	2.0681	0.296	6.993	0.000
V1	3.0967	0.182	17.045	0.000
V2	-2.1940	0.163	-13.475	0.000
V3	1.5009	0.148	10.155	0.000
V4	-0.4003	0.165	-2.422	0.018
V5	0.3072	0.156	1.965	0.053
V6	-0.0531	0.148	-0.359	0.721
V7	-0.0174	0.180	-0.096	0.923
V8	-0.1318	0.181	-0.728	0.469
V9_G2	3.7358	0.389	9.609	0.000
V9_G3	-3.1683	0.447	-7.092	0.000
V3sq	1.2174	0.088	13.846	0.000

> **參考答案**：殘差對擬合值圖沒有明顯的趨勢，但殘差對 V_3 的散佈圖呈明顯的 **U 形**，表示模型遺漏了 V_3 的非線性項（V_3 只是決定擬合值的眾多變數之一，所以在殘差對擬合值圖中被淹沒）。加入 V_3^2 後，其係數約 1.22、p 值 1.0e-22，極為顯著；adjusted R^2 由 0.702 提高到 0.913，測試 MSE 由 10.55 降到 2.55。

(d) 使用訓練集，以逐步向前選取法（forward stepwise selection）從空模型開始，每一步加入使 RSS 最小的變數，直到 11 個變數全部加入；再從這條路徑上的模型中分別以 AIC 與 BIC 選出最佳模型。請列出變數加入的順序，以及兩個準則所選的變數與其測試集 MSE。哪一個準則選出的模型較小？請從兩者的懲罰項說明原因。 (10%)

```
models1 = pd.DataFrame(columns=['RSS', 'model'])
predictors = []
for i in range(1, len(X_train2.columns) + 1):
    models1.loc[i] = forward(X_train2, y_train, predictors)
    exog = models1.loc[i]['model'].model.exog_names.copy()
    exog.remove('const')
    predictors = exog
    print(i, predictors)
```

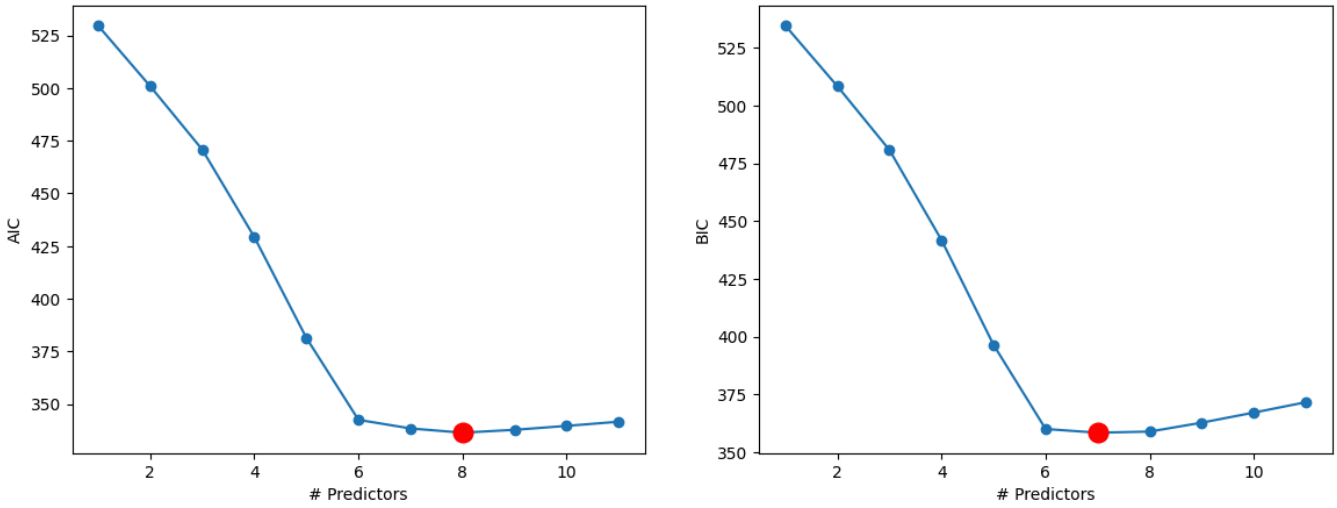
```
1 ['V1']
2 ['V1', 'V9_G2']
3 ['V1', 'V9_G2', 'V2']
4 ['V1', 'V9_G2', 'V2', 'V3sq']
5 ['V1', 'V9_G2', 'V2', 'V3sq', 'V3']
6 ['V1', 'V9_G2', 'V2', 'V3sq', 'V3', 'V9_G3']
7 ['V1', 'V9_G2', 'V2', 'V3sq', 'V3', 'V9_G3', 'V4']
8 ['V1', 'V9_G2', 'V2', 'V3sq', 'V3', 'V9_G3', 'V4', 'V5']
9 ['V1', 'V9_G2', 'V2', 'V3sq', 'V3', 'V9_G3', 'V4', 'V5', 'V8']
10 ['V1', 'V9_G2', 'V2', 'V3sq', 'V3', 'V9_G3', 'V4', 'V5', 'V8', 'V6']
11 ['V1', 'V9_G2', 'V2', 'V3sq', 'V3', 'V9_G3', 'V4', 'V5', 'V8', 'V6', 'V7']
```

```
aic_f, bic_f = [], []
for m in models1.model:
    aic_f.append(m.aic)
    bic_f.append(m.bic)
k_aic = np.array(aic_f).argmin() + 1      # models1 的 index 從 1 開始
k_bic = np.array(bic_f).argmin() + 1
final_aic, final_bic = models1.model[k_aic], models1.model[k_bic]
feat_aic = final_aic.params.index.drop('const').tolist()
feat_bic = final_bic.params.index.drop('const').tolist()
mse_aic = mean_squared_error(y_test, final_aic.predict(add_constant(X_test2[feat_aic])))
mse_bic = mean_squared_error(y_test, final_bic.predict(add_constant(X_test2[feat_bic])))
order = [models1.model[i].params.index[-1] for i in range(1, len(X_train2.columns) + 1)]
R('d_order', order)
R('d_k', k_aic); R('d_sel', feat_aic); R('d_mse', mse_aic, 2)
R('d_kb', k_bic); R('d_selb', feat_bic); R('d_mseb', mse_bic, 2)
R('d_extra', [v for v in feat_aic if v not in feat_bic])
R('d_logn', np.log(len(y_train)), 1)
```

```
forward 加入順序：V1, V9_G2, V2, V3sq, V3, V9_G3, V4, V5, V8, V6, V7
AIC 選出的變數個數：8
AIC 選出的變數：V1, V9_G2, V2, V3sq, V3, V9_G3, V4, V5
AIC 模型測試 MSE：2.46
BIC 選出的變數個數：7
BIC 選出的變數：V1, V9_G2, V2, V3sq, V3, V9_G3, V4
BIC 模型測試 MSE：2.66
AIC 多選而 BIC 未選的變數：V5
log n（訓練集）：4.5
```

```
4.499809670330265
```

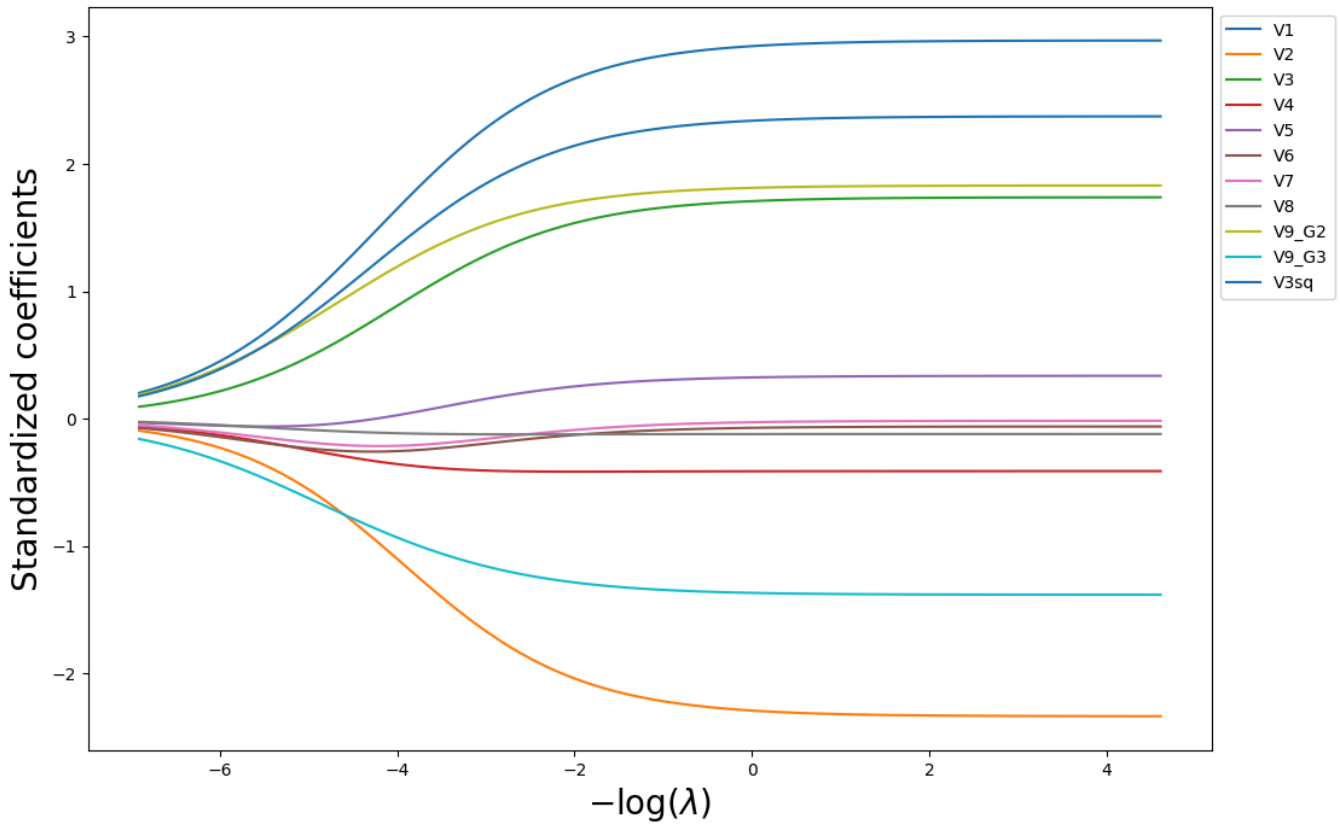
```
fig, axes = subplots(1, 2, figsize=(14, 5))
for ax, crit, k, lab in zip(axes, [aic_f, bic_f], [k_aic, k_bic], ['AIC', 'BIC']):
    ax.plot(range(1, 12), crit, '-o'); ax.plot(k, min(crit), 'or', ms=12)
    ax.set_xlabel('# Predictors'); ax.set_ylabel(lab)
```



> **參考答案**：加入順序：V1, V9_G2, V2, V3sq, V3, V9_G3, V4, V5, V8, V6, V7。AIC 選出 8 個變數：V1, V9_G2, V2, V3sq, V3, V9_G3, V4, V5，測試 MSE 2.46；BIC 選出 7 個變數：V1, V9_G2, V2, V3sq, V3, V9_G3, V4，測試 MSE 2.66。BIC 選出的模型較小（少了 V5）：AIC 對每多一個參數的懲罰是 2，BIC 是 $\log n$，本題訓練集 $n = 90$，$\log n \approx 4.5 > 2$，所以 BIC 對加入變數的要求較嚴格，通常選出較小的模型。較小的模型不一定在單一測試集上表現較好，本題兩者的測試 MSE 相近。

(e) 在訓練集上擬合 Ridge regression，繪製 11 個自變數標準化後的迴歸係數隨 λ（sklearn 的 `alpha`）變化的路徑圖。λ 由 10^3 掃描到 10^{-2}，以對數刻度均勻取 100 個點；以 10 折交叉驗證選出使**平均驗證 MSE 最小**的 λ，回報訓練集與測試集 MSE。是否有係數被估計為 0？ (8%)

```
lambdas = 10**np.linspace(3, -2, 100)
scaler = StandardScaler()
Xs = scaler.fit_transform(X_train2)
ridge = skl.Ridge()
coefs = []
for a in lambdas:
    ridge.set_params(alpha=a)
    ridge.fit(Xs, y_train)
    coefs.append(ridge.coef_)
soln_path = pd.DataFrame(np.array(coefs), columns=X_train2.columns, index=-np.log(lambdas))
path_fig, ax = subplots(figsize=(12, 8))
soln_path.plot(ax=ax, legend=False)
ax.set_xlabel(r'$-\log(\lambda)$', fontsize=20); ax.set_ylabel('Standardized coefficients',
fontsize=20)
ax.legend(loc='upper left', bbox_to_anchor=(1, 1));
```



```
kfold = skm.KFold(10, shuffle=True, random_state=2026)
ridgeCV = skl.RidgeCV(alphas=lambdas, cv=kfold, scoring='neg_mean_squared_error')   # 預設 sc
oring 為 R^2
pipeCV = Pipeline(steps=[('scaler', StandardScaler()), ('ridge', ridgeCV)])
pipeCV.fit(X_train2, y_train)
tuned_ridge = pipeCV.named_steps['ridge']
mse_tr = mean_squared_error(y_train, pipeCV.predict(X_train2))
mse_te = mean_squared_error(y_test, pipeCV.predict(X_test2))
R('e_alpha', tuned_ridge.alpha_, 3); R('e_tr', mse_tr, 2); R('e_te', mse_te, 2)
R('e_min', np.abs(tuned_ridge.coef_).min(), 3)
print(tuned_ridge.alpha_); pd.Series(tuned_ridge.coef_, index=X_train2.columns).round(3)
```

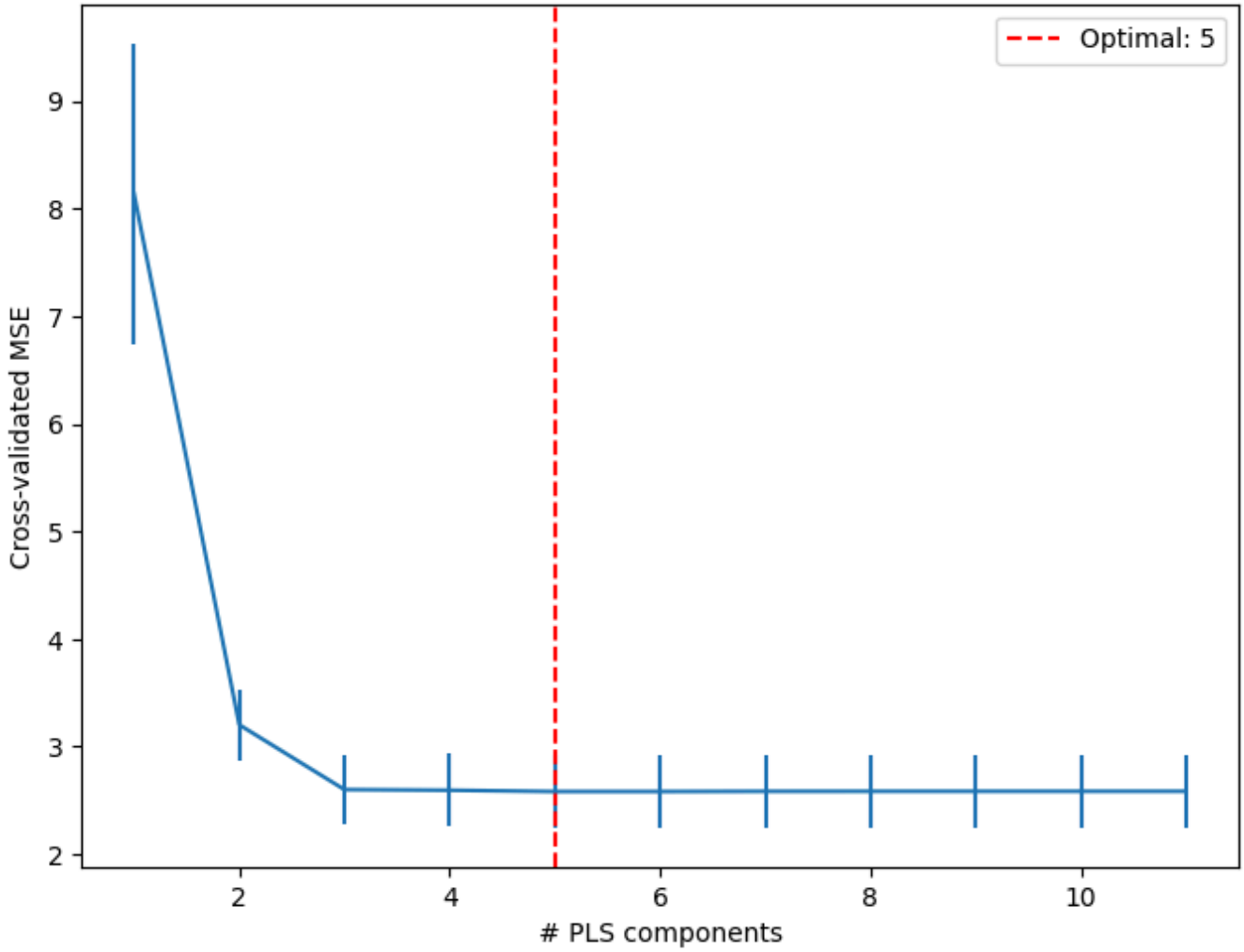
```
CV 選出的 λ：0.521
Ridge 訓練 MSE：2.00
Ridge 測試 MSE：2.61
Ridge 係數絕對值最小值：0.023
0.5214008287999684
```

```
V1       2.945
V2      -2.312
V3       1.722
V4      -0.412
V5       0.329
V6      -0.067
V7      -0.023
V8      -0.121
V9_G2    1.820
V9_G3   -1.374
V3sq     2.355
dtype: float64
```

> **參考答案**：CV 選出 $\lambda \approx 0.521$，訓練 MSE 2.00、測試 MSE 2.61。沒有任何係數恰為 0（最小的絕對值約 0.023）：ridge 的 ℓ_2 懲罰通常只會把係數往 0 收縮，不會像 lasso 的 ℓ_1 懲罰那樣產生稀疏解（部分係數恰為 0），因此不做變數選擇。

(f) 在訓練集上擬合偏最小平方迴歸（PLS，Partial Least Squares），以 10 折交叉驗證從 $M = 1, \ldots, 11$ 中選擇成分數 M，並繪製 CV MSE 對 M 的圖形。回報所選的 M 與訓練集、測試集 MSE。PLS 與 PCR 在建構成分（directions）時有什麼不同？ (8%)

```
pls = PLSRegression(n_components=2, scale=True)
param_grid = {'n_components': range(1, 12)}
grid = skm.GridSearchCV(pls, param_grid, cv=kfold, scoring='neg_mean_squared_error')
grid.fit(X_train2, y_train)
n_comp = param_grid['n_components']
pls_fig, ax = subplots(figsize=(8, 6))
ax.errorbar(n_comp, -grid.cv_results_['mean_test_score'], grid.cv_results_['std_test_score']
/ np.sqrt(10))
ax.set_xlabel('# PLS components'); ax.set_ylabel('Cross-validated MSE')
best_n_comp = n_comp[np.argmin(-grid.cv_results_['mean_test_score'])]
ax.axvline(best_n_comp, ls='--', c='r', label=f'Optimal: {best_n_comp}'); ax.legend();
```




```
best_pls = grid.best_estimator_
R('f_M', best_n_comp); R('f_cv', np.round(-grid.cv_results_['mean_test_score'][:5], 2).tolis
t(), 2)
R('f_tr', mean_squared_error(y_train, best_pls.predict(X_train2)), 2)
R('f_te', mean_squared_error(y_test, best_pls.predict(X_test2)), 2)
np.round(-grid.cv_results_['mean_test_score'], 3)
```

```
CV 選出的 PLS 成分數 M：5
M=1–5 的 CV MSE：8.14, 3.20, 2.60, 2.59, 2.58
PLS 訓練 MSE：2.00
PLS 測試 MSE：2.55
```

```
array([8.138, 3.203, 2.601, 2.595, 2.583, 2.583, 2.585, 2.585, 2.585,
       2.586, 2.586])
```

> **參考答案**：CV 選出 $M = 5$（前五個 CV MSE：8.14, 3.20, 2.60, 2.59, 2.58，$M \geq 3$ 後幾乎持平），訓練 MSE 2.00、測試 MSE 2.55。PCR 的方向只由 X 的變異決定（非監督式）；PLS 在建構每個成分時會使用 y，第一個方向是讓與 y 的共變異最大的線性組合（監督式），因此通常用較少的成分就能達到相近的誤差。

題目二：使用 "Data2" 資料集進行三類別分類分析

請使用 **"Data2"** 資料集，其中應變數 $y \in \{0, 1, 2\}$（三個類別），自變數為 $V_1, \ldots, V_8$。請以**加權 F1 分數（weighted F1-score）**作為評估指標：

$$F1_{\text{weighted}} = \sum_{i=1}^{K} \frac{N_i}{N} F1_i, \qquad F1_i = \frac{2 \cdot \text{precision}_i \cdot \text{recall}_i}{\text{precision}_i + \text{recall}_i}$$

其中 N_i 為第 i 類的真實樣本數。可參考 `sklearn.metrics` 模組中的方法計算。

- 當使用具有隨機性的演算法（例如隨機抽樣）時，請設定 `random_state=2026` 以確保結果可重現。
- 若所選模型**不具尺度不變性（scale-invariance）**，請在建模前對輸入的自變數進行標準化（standardization）。

```
def f1w(y_true, y_pred):
    return metrics.f1_score(y_true, y_pred, average='weighted')
```

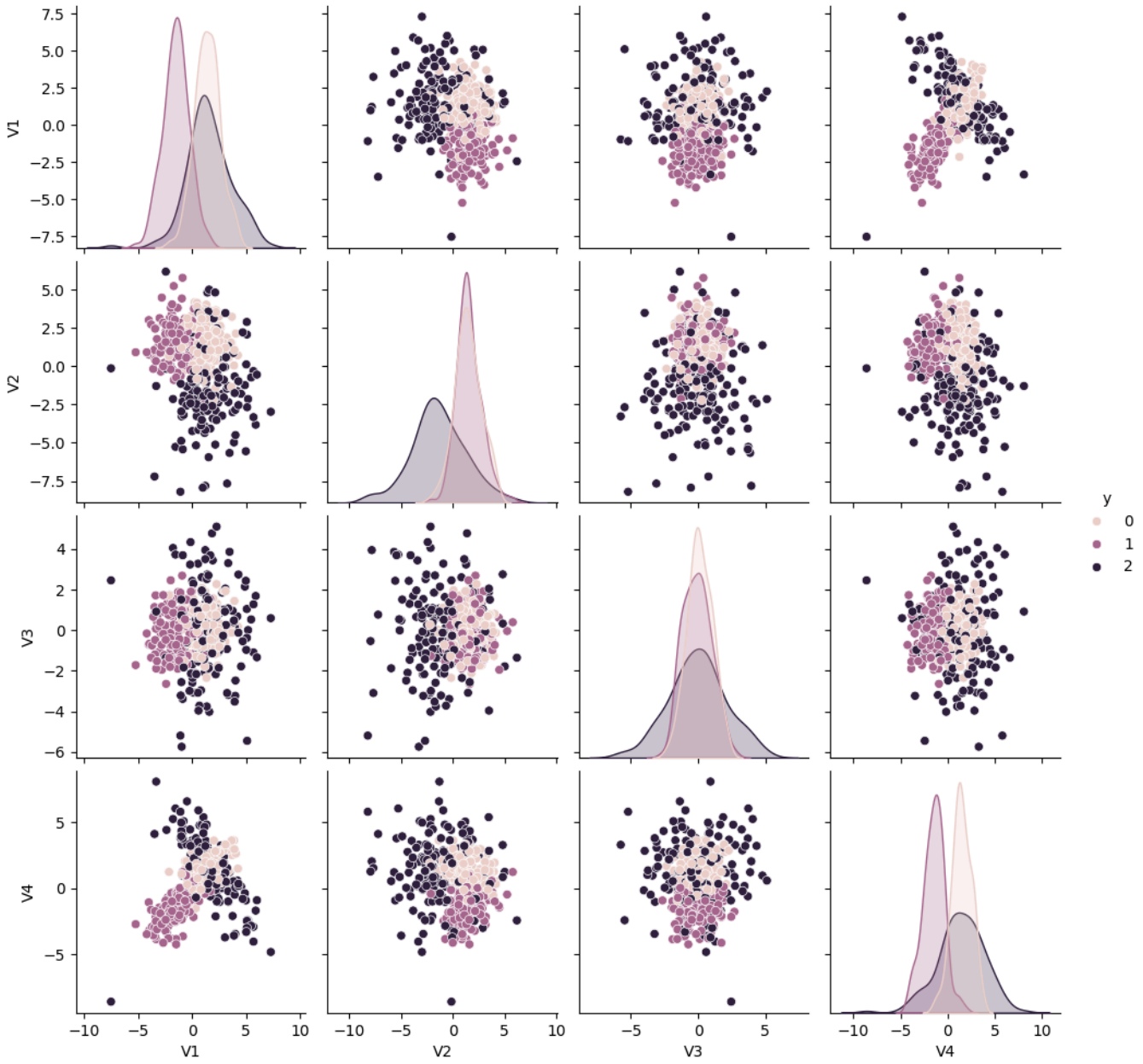
(a) 使用 `train_test_split` 以 `test_size=0.3` 切分資料，統計訓練集與測試集中各類別的樣本數。以依類別著色的 `pairplot` 繪製 V_1–V_4 的成對散佈圖，各類別的分散程度（共變異數）看起來是否相同？ (5%)

```
df2 = pd.read_csv('Data2.csv')
X2 = df2.drop(columns='y'); y2 = df2['y']
X2_train, X2_test, y2_train, y2_test = train_test_split(X2, y2, test_size=0.3, random_state=
2026)
R('a2_tr', y2_train.value_counts().sort_index().tolist()); R('a2_te', y2_test.value_counts
().sort_index().tolist())
y2_train.value_counts().sort_index(), y2_test.value_counts().sort_index()
```

```
訓練集類別 0／1／2 筆數：103, 106, 106
測試集類別 0／1／2 筆數：45, 43, 47
```

```
(y
 0    103
 1    106
 2    106
 Name: count, dtype: int64,
 y
 0    45
 1    43
 2    47
 Name: count, dtype: int64)
```

```
sns.pairplot(df2[['V1', 'V2', 'V3', 'V4', 'y']], hue='y');
```




```
R('a2_sd', df2.groupby('y')[['V1', 'V2', 'V3', 'V4']].std().mean(axis=1).round(2).tolist(), 2)
df2.groupby('y')[['V1', 'V2', 'V3', 'V4']].std().round(2)
```

各類 V1–V4 標準差平均 (類別 0, 1, 2)：1.10, 1.15, 2.33

y	V1	V2	V3	V4
0	1.18	1.27	0.93	1.03
1	1.20	1.21	1.05	1.12
2	2.08	2.63	2.12	2.49

> **參考答案**：訓練集類別 0 / 1 / 2 為 103, 106, 106，測試集為 45, 43, 47，三類大致平衡。由散佈圖可見類別 2 的點分布得比類別 0、1 分散許多（各類 V_1–V_4 標準差平均為 1.10, 1.15, 2.33），三類的共變異數並不相同。

(b) 使用訓練集建立線性判別分析（LDA）分類器，回報訓練集與測試集的加權 F1 分數，並列出測試集的混淆矩陣。 (7%)

```
lda = LDA(store_covariance=True).fit(X2_train, y2_train)
R('b2_tr', f1w(y2_train, lda.predict(X2_train))); R('b2_te', f1w(y2_test, lda.predict(X2_tes
t)))
print(confusion_table(lda.predict(X2_test), y2_test))
float(RES['b2_tr']), float(RES['b2_te'])
```

```
LDA 訓練加權 F1：0.878
LDA 測試加權 F1：0.779
Truth       0   1   2
Predicted
0          31   2  13
1           2  40   0
2          12   1  34
(0.878, 0.779)
```

> **參考答案**：LDA：訓練 0.878、測試 0.779。

(c) 使用訓練集建立二次判別分析（QDA）分類器，回報訓練集與測試集的加權 F1 分數，並列出測試集的混淆矩陣。 (7%)

```
qda = QDA(store_covariance=True).fit(X2_train, y2_train)
R('c2_tr', f1w(y2_train, qda.predict(X2_train))); R('c2_te', f1w(y2_test, qda.predict(X2_tes
t)))
print(confusion_table(qda.predict(X2_test), y2_test))
float(RES['c2_tr']), float(RES['c2_te'])
```

```
QDA 訓練加權 F1：0.987
QDA 測試加權 F1：0.978
Truth       0   1   2
Predicted
0          44   1   0
1           0  41   0
2           1   1  47
(0.987, 0.978)
```

> **參考答案**：QDA：訓練 0.987、測試 0.978。

(d) 使用訓練集建立高斯樸素貝氏分類器（Gaussian Naive Bayes），回報訓練集與測試集的加權 F1 分數。Naive Bayes 對類別內各自變數的關係做了什麼假設？ (7%)

```
NB = GaussianNB().fit(X2_train, y2_train)
R('d2_tr', f1w(y2_train, NB.predict(X2_train))); R('d2_te', f1w(y2_test, NB.predict(X2_tes
t)))
float(RES['d2_tr']), float(RES['d2_te'])
```

```
NB 訓練加權 F1：0.917
NB 測試加權 F1：0.897
(0.917, 0.897)
```

> **參考答案**：Naive Bayes：訓練 0.917、測試 0.897。它假設在給定類別下，各自變數**彼此獨立**（等價於每一類的共變異數矩陣是對角矩陣），但允許各類別有不同的變異數。

(e) 使用標準化後的自變數建立 KNN 分類器，以 10 折交叉驗證、加權 F1 為準則，從 $K \in \{1, 3, 5, \ldots, 29\}$ 中選出最佳 K。回報所選的 K 與訓練集、測試集的加權 F1 分數。 (9%)

```
scaler2 = StandardScaler().fit(X2_train)
Z_train, Z_test = scaler2.transform(X2_train), scaler2.transform(X2_test)
kfold10 = skm.KFold(10, shuffle=True, random_state=2026)
grid_knn = skm.GridSearchCV(KNeighborsClassifier(), {'n_neighbors': range(1, 30, 2)}, cv=kfo
ld10, scoring='f1_weighted')
grid_knn.fit(Z_train, y2_train)
knn = grid_knn.best_estimator_
R('e2_K', grid_knn.best_params_['n_neighbors'])
R('e2_tr', f1w(y2_train, knn.predict(Z_train))); R('e2_te', f1w(y2_test, knn.predict(Z_tes
t)))
int(RES['e2_K']), float(RES['e2_tr']), float(RES['e2_te'])
```

```
CV 選出的 K：3
KNN 訓練加權 F1：0.949
KNN 測試加權 F1：0.853
```

```
(3, 0.949, 0.853)
```

> **參考答案**：CV 選出 $K = 3$，訓練 0.949、測試 0.853。

(f) 比較 (b)–(e) 四個模型的測試集加權 F1 分數。哪一個模型最好？請從「各類別的共變異數是否相同」的角度解釋 LDA、QDA 與 Naive Bayes 表現的差異。 (5%)

```
pd.Series({'LDA': RES['b2_te'], 'QDA': RES['c2_te'], 'NB': RES['d2_te'], 'KNN': RES['e2_t
e']}).astype(float)
```

```
LDA    0.779
QDA    0.978
NB     0.897
KNN    0.853
dtype: float64
```

> **參考答案**：QDA 最好（0.978），其次為 Naive Bayes（0.897）、KNN（0.853），LDA 最差（0.779）。LDA 假設所有類別共用同一個共變異數矩陣，決策邊界是線性的；但 (a) 已看到類別 2 比其他兩類分散得多，各類共變異數不同。若各類的條件分布可用「共變異數不同的多變量常態」近似，QDA 的二次邊界較能反映這種結構，本次測試結果也支持 QDA。Naive Bayes 也允許各類別有不同的變異數，只是忽略變數間的相關，因此介於兩者之間。

題目三：交叉驗證的正確與錯誤做法

本部分探討交叉驗證（cross-validation）的正確與錯誤做法。

(a) 若先使用**全部資料**挑出與 y 相關性最高的自變數，再用這些自變數做 k 折交叉驗證來估計分類器的測試錯誤率，為什麼會低估真正的測試錯誤率？正確的流程中，「挑選自變數」這一步應該放在哪裡？ (6%)

> **參考答案**：挑選變數時已經用到了每一個驗證折（validation fold）的 y；之後再做 CV 時，驗證折並不是模型「沒看過」的資料，於是錯誤率被低估（資料洩漏，data leakage）。正確流程是把「挑選變數」視為模型配適的一部分：在每一折中，只用該折的訓練部分挑選變數並配適模型，再用該折的驗證部分計算錯誤率。

(b) 執行以下模擬：在每一次重複中產生 $n = 60$ 筆、$p = 500$ 個與 y **完全無關**的自變數，以及隨機的二元 y：

```
for i in range(20):
    rng = np.random.default_rng(2026 + i)
    X = rng.normal(size=(60, 500))
    y = rng.integers(0, 2, size=60)
```

可使用下列函式挑出 $|\mathrm{corr}(X_j, y)|$ 最大的 k 個自變數：

```
def top_k(X, y, k=10):
    corr = np.array([
        np.corrcoef(X[:, j], y)[0, 1]
        for j in range(X.shape[1])
    ])
    return np.argsort(np.abs(corr))[::-1][:k]
```

以挑出的 10 個自變數建立 `LogisticRegression()`（預設參數），並以 `KFold(n_splits=5, shuffle=True, random_state=2026)` 計算 5 折 CV 錯誤率：（i）錯誤做法：先用全部 60 筆資料挑選 10 個變數，再做 CV；（ii）正確做法：在每一折中只用訓練部分挑選 10 個變數。重複 20 次，回報兩種做法的平均 CV 錯誤率，並與真正的測試錯誤率比較（因為 X 與 y 無關，真正的錯誤率為多少？）。 (14%)

```
def top_k(X, y, k=10):
    corr = np.array([
        np.corrcoef(X[:, j], y)[0, 1]
        for j in range(X.shape[1])
    ])
    return np.argsort(np.abs(corr))[::-1][:k]

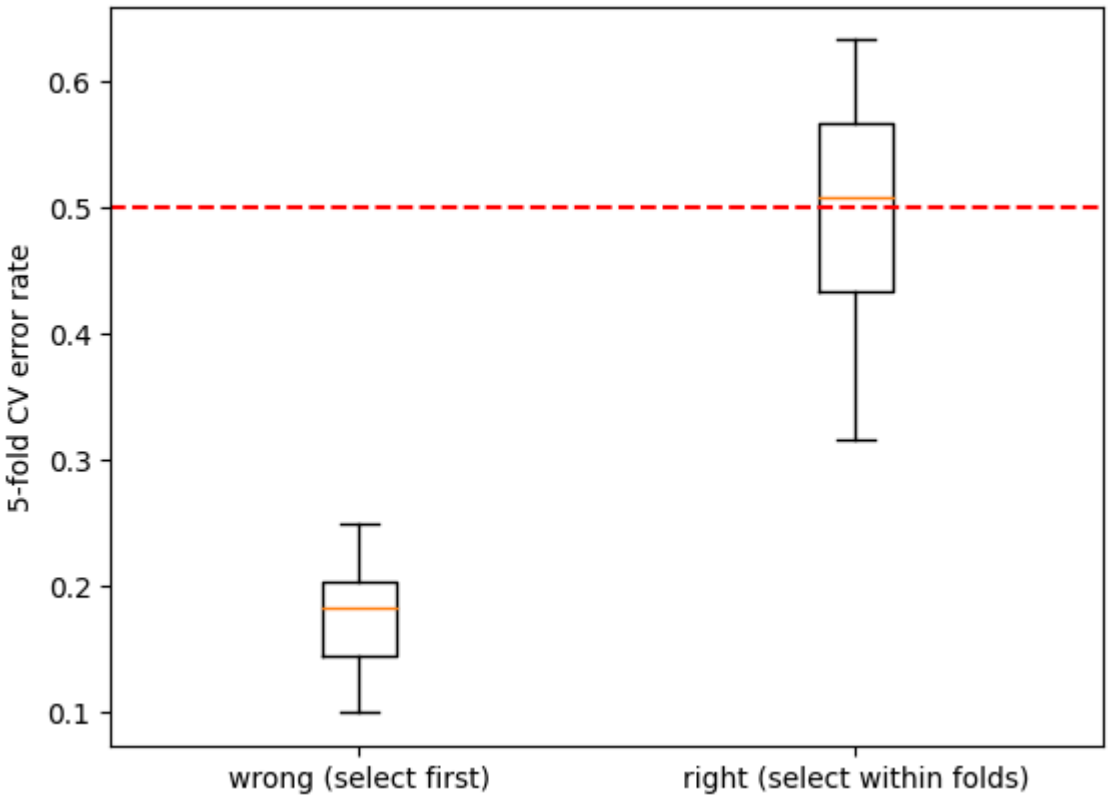
kf5 = KFold(n_splits=5, shuffle=True, random_state=2026)
wrong, right = [], []
for i in range(20):
    rng = np.random.default_rng(2026 + i)
    X = rng.normal(size=(60, 500))
    y = rng.integers(0, 2, size=60)
    sel_all = top_k(X, y)                          # (i) 先用全部資料挑變數
    err_w, err_r = [], []
    for tr, va in kf5.split(X):
        m = LogisticRegression().fit(X[tr][:, sel_all], y[tr])
        err_w.append(np.mean(m.predict(X[va][:, sel_all]) != y[va]))
```

```
        sel = top_k(X[tr], y[tr])                # (ii) 每一折內只用訓練部分挑變數
        m = LogisticRegression().fit(X[tr][:, sel], y[tr])
        err_r.append(np.mean(m.predict(X[va][:, sel]) != y[va]))
    wrong.append(np.mean(err_w)); right.append(np.mean(err_r))
R('p3_wrong', np.mean(wrong)); R('p3_right', np.mean(right))
np.mean(wrong), np.mean(right)
```

```
錯誤做法平均 CV 錯誤率：0.176
正確做法平均 CV 錯誤率：0.500
```

```
(0.17583333333333334, 0.5)
```

```
plt.boxplot([wrong, right], labels=['wrong (select first)', 'right (select within folds)'])
plt.axhline(0.5, c='r', ls='--'); plt.ylabel('5-fold CV error rate');
```



> **參考答案**：錯誤做法的平均 CV 錯誤率約 0.176，正確做法約 0.500。因為 X 與 y 無關，任何分類器的真正錯誤率都是 0.5（等同猜硬幣）。錯誤做法嚴重低估錯誤率，讓人誤以為找到了有預測力的變數；正確做法的估計接近 0.5。